

SENTIMENT ANALYSIS - CHAPTER 8

- SUBFIELD OF NATURAL LANGUAGE PROCESSING (NLP) → SENTIMENT ANALYSIS → USE MACHINE LEARNING ALGORITHM TO CLASSIFY DOCUMENTS BASED ON SENTIMENT.
- TEST WITH IMDB

IMDB MOVIE REVIEW DATA FOR TEXT PROCESSING

- SENTIMENT ANALYSIS → SOMETIMES CALLED OPINION MINING
- POPULAR TASK → CLASSIFICATION OF DOCUMENTS BASED ON EXPRESSED OPINION OR EMOTION OF AUTHOR

BAG-OF-WORDS MODEL

- BAG-OF-WORDS MODEL → ALLOW US TO REPRESENT TEXT AS NUMERICAL FEATURE VECTORS.
 - ↳ SUMMARISED:
 1. CREATE VOCABULARY OF UNIQUE TOKENS - E.G. WORDS FROM ENTIRE SET OF DOCUMENTS
 2. CONSTRUCT FEATURE VECTOR FROM EACH DOCUMENT → THE COUNT OF HOW OFTEN EACH WORD OCCURS.
- SINCE UNIQUE WORDS IN EACH DOCUMENT ONLY REPRESENT SMALL SUBSET → FEATURE VECTOR WILL MAINLY CONSIST OF 0 → CALL THEM SPARSE

WORDS TO FEATURE VECTORS

- RAW TERM FREQUENCIES - $tf(t, d)$ - NUMBER OF TIMES TERM t , OCCURS IN DOCUMENT, d → FUNDAMENTAL METRIC IN NLP.
- IN BAG-OF-WORDS MODEL → WORD/TERM ORDER IN SENTENCE/DOCUMENT DOESN'T MATTER

N-GRAM MODEL

- SEQUENCE OF ITEMS IN BAG-OF-WORDS → CALLED 1-GRAM OR UNIGRAM MODEL. → EACH TOKEN IN VOCAB REPRESENTS A SINGLE WORD.
- CONTIGUOUS SEQUENCE OF ITEMS IN NLP → CALLED N-GRAMS.
- CHOICE OF NUMBER, n DEPENDS ON APPLICATION.
- E.G.:
 - 1-GRAM: "the", "sun", "is", "shining"
 - 2-GRAM: "the sun", "sun is", "is shining"
- n CAN BE SWITCHED BY "ngram-range" IN CountVectorizer

ASSESSING WORD RELEVANCE VIA TERM FREQUENCY-INVERSE DOCUMENT FREQUENCY

- TERM FREQUENCY-IN-VERSE DOCUMENT FREQUENCY (TF-IDF) → USE TO DOWNWEIGHT FREQUENTLY OCCURRING WORDS IN FEATURE VECTORS.

↳ PRODUCT OF TERM FREQUENCY + INVERSE DOCUMENT FREQUENCY

$$tf-idf(t, d) = tf(t, d) \times idf(t, d)$$

↳ TERM FREQUENCY ↳ INVERSE DOCUMENT FREQUENCY

$$idf(t, d) = \log \frac{n_d}{1 + df(d, t)}$$

n_d = TOTAL NUMBER OF DOCUMENTS

$df(d, t)$ = NUMBER OF DOCUMENTS, d , CONTAIN THE TERM, t

1 = OPTIONAL → PURPOSE IS TO ASSIGNING A NON-ZERO VALUE TO TERMS THAT OCCUR IN NONE OF THE TRAINING EXAMPLES

log = ENSURED FOR LOW DOCUMENT FREQUENCIES ARE NOT GIVEN TOO MUCH WEIGHT.

SCIKIT-LEARN → 'TfidfTransformer' → INVERSE DOCUMENT FREQUENCY: $idf(t, d) = \log \frac{1 + n_d}{1 + df(d, t)}$

↳ $tf-idf$ → SCIKIT-LEARN → $tf-idf(t, d) = tf(t, d) \times (idf(t, d) + 1)$

- NORMALLY → NORMALISE RAW TERM FREQUENCIES BEFORE CALCULATING $tf-idf$ → 'TfidfTransformer' NORMALISES $tf-idf$ s. DEFAULT norm 'l2'
- RETURNS VECTOR LENGTH 1 BY UNNORMALISED VECTOR, $\sqrt{L2-NORM}$:

$$v_{norm} = \frac{v}{\|v\|_2} = \frac{v}{\sqrt{v_1^2 + v_2^2 + \dots + v_n^2}} = \frac{v}{(\sum_{i=1}^n v_i^2)^{1/2}}$$

CLEANING TEXT DATA

- FIRST IMPORTANT STEP → CLEAN TEXT DATA BY STRIPPING OF ALL UNWANTED CHARACTERS.
- ACCOMPLISH THIS WITH REGEX → REGULAR EXPRESSION
- REMOVING HTML MARKUP → REGEX ACCEPTABLE → WON'T USE HTML MARKUP FURTHER

PROCESSING DOCUMENTS INTO TOKENS

- TOKENISE DOCUMENTS → SPLIT INDIVIDUAL WORDS → SPLIT CLEANED DOCUMENT AFTER WHITESPACE CHARACTERS
- ANOTHER TECHNIQUE → **WORD STEMMING** → TRANSFORMING WORDS TO THEIR ROOT FORM. → **ALLOWS US TO MAP RELATED WORDS TO THE SAME STEM.**
 - ↳ ORIGINAL ALGORITHM BY MARTIN F. PORTER (1979) → KNOWN AS **PORTER STEMMER ALGORITHM**

STEMMING ALGORITHMS

- PORTER STEMMING ALGORITHM → OLDEST + SIMPLEST STEMMING ALSO
- OTHER EXAMPLES:
 - SNOWBALL STEMMER (PORTER2 OR ENGLISH STEMMER)
 - LANCASTER STEMMER (PAICE/HUSK STEMMER) } **BOTH FASTER THAN ORIGINAL PORTER STEMMER**
 - ↳ **AGGRESSIVE COMPARED TO PORTER → PRODUCES SHORTER + MORE OBSCURE WORDS**
- STEMMING CAN CREATE NON-REAL WORDS (E.G. 'THU' INSTEAD OF 'THIS')
 - ↳ **LEMMAIZATION** → **TECHNIQUE TO OBTAIN CANONICAL (GRAMMATICALLY CORRECT) FORMS FOR INDIVIDUAL WORDS ('LEMMAS')**
 - ↳ **COMPUTATIONALLY MORE EXPENSIVE + DIFFICULT → NOT WORTH THE TIME + EFFORT BECAUSE IT HAS LITTLE TO NO EFFECT ON TEXT CLASSIFICATION**
- **STOP WORD REMOVAL** → **EXTREMELY COMMON WORDS IN ALL SORTS OF TEXT THAT BEAR LITTLE TO NO USEFUL INFORMATION FOR DISTINGUISHING DIFFERENT CLASSES.**
 - ↳ CAN BE USEFUL → IF WORKING WITH RAW OR NORMALIZED TERM FREQUENCIES, RATHER THAN TF-IDFS.

- **NATIVE BAYES CLASSIFIER** → **VERY POPULAR TEXT CLASSIFICATION CLASSIFIER → EASY TO IMPLEMENT + COMPUTATIONALLY EFFICIENT + PERFORMS WELL ON SOME DATASETS.**
 - ↳ **GAINED POPULARITY THROUGH EMAIL SPAM FILTERING** (MENTION I HAVE OTHER NOTE ON NATIVE BAYES CLASSIFIER)

WORKING WITH BIGGER DATA - ONLINE ALGORITHM + OUT-OF-CORE LEARNING

- COMPUTATIONALLY EXPENSIVE TO CONSTRUCT FEATURE VECTOR FOR LARGE DATASET.
- **REAL-WORLD APPLICATION** → **UNCOMMON TO WORK WITH LARGE DATASETS THAT EXCEED COMPUTER'S MEMORY.**
- **OUT-OF-CORE LEARNING** → **ALLOWS US TO WORK WITH LARGE DATASETS → FIT CLASSIFIER INCREMENTALLY ON SMALLER BATCHES**
- LATER → WE WILL USE NEURAL NETWORKS WITH OUT-OF-CORE LEARNING + STOCHASTIC GRADIENT DESCENT.
- OUT-OF-CORE LEARNING → MEMORY EFFICIENT.

WORD2VEC MODEL

- MODERN ALTERNATIVE TO BAG-OF-WORDS MODEL.
- RELEASED BY GOOGLE IN 2013
- **UNSUPERVISED LEARNING ALGORITHM → BASED ON NEURAL NETWORKS → ATTEMPT TO AUTOMATICALLY LEARN RELATIONSHIP BETWEEN WORDS.**
- IDEA → PUTS SIMILAR WORDS IN SMALL CLUSTERS - VIA CLEVER VECTOR SPACING

TOPIC MODELLING WITH LATENT DIRICHLET ALLOCATION

- **TOPIC MODELLING** → **BROAD TASK OF ASSIGNING TOPICS TO UNLABELLED TEXT DOCUMENTS.**
 - ↳ TYPICAL EXAMPLE → CATEGORISATION OF DOCUMENTS IN LARGE CORPUS OF NEWSPAPER ARTICLES.
 - CAN BE CLASSIFIED AS **CLUSTERING TASK (UNSUPERVISED LEARNING)**
- POPULAR TECHNIQUE → **LATENT DIRICHLET ALLOCATION (LDA)** → **DON'T CONFUSE WITH LINEAR DISCRIMINANT ANALYSIS**

DECOMPOSING TEXT DOCUMENTS WITH LDA

- **LDA** → **PROBABILISTIC MODEL → TRIES TO FIND GROUPS OF WORDS THAT APPEAR TOGETHER FREQUENTLY ACROSS DIFFERENT DOCUMENTS.**
- INPUT TO LDA IS BAG-OF-WORDS MODEL
 - ↳ SPLITS MATRIX INPUT TO TWO NEW MATRICES:
 - **DOCUMENT-TO-TOPIC MATRIX**
 - **WORD-TO-TOPIC MATRIX**
- **LDA DECOMPOSITION** BAG-OF-WORDS MODEL IN A WAY THAT IF WE MULTIPLY THE TWO NEW MATRICES WE GET THE BAG-OF-WORDS INPUT MATRIX
- **ONLY DOWNSIDE** → **WE MUST DEFINE NUMBER OF TOPICS BEFOREHAND → LDA HYPERPARAMETER.**

EXPECTATION-MAXIMIZATION

- SKIKIT-LEARN LDA USES EXPECTATION-MAXIMIZATION (EM)